

NineToothed 算子开发 Skill

T3-1-1 决赛赛题报告

Skill	ninetoothed-operator-dev-skill
赛事	2026 春季启元人工智能大赛
赛题号	T3-1-1
参赛者	刘李宏
小组名称	123123
GitHub ID	Dreamt-Deer-Waking-Fish
报告日期	2026-07-12

5	2	4
自测案例	真实 benchmark	comparison patch checks

签名版声明

参赛者身份、小组名称、GitHub ID 与签名日期已写入 Honor Code。技术结论、benchmark 数值、性能回退说明和限制声明保持不变。

目录

目录 2

1. 摘要 3

2. 赛题理解与设计原则 3

3. 适用与不适用范围 3

4. 包结构 4

5. 一次成功导向的核心 workflow 4

6. Elementwise/Broadcast 自测 4

7. Reduction/Blocking 自测 5

8. Layout-Sensitive 自测 5

9. Performance/AOT/Diagnosis 自测 5

10. 真实实现型自测 5

11. Benchmark 证据 6

12. 失败诊断与修复闭环 6

 Patch 行尾与上下文 6

 服务器 patch 失败 6

 Layout helper 失败与修复 6

 InfiniCore native import 6

13. No-Skill 与 Skill-Enabled 对比 6

14. 安全、依赖、授权、引用与 AI 披露 6

15. 局限性与维护计划 7

16. 评分项追踪矩阵 7

结论 7

1. 摘要

本项目交付一个面向 NineToothed/ntops 真实开发任务的离线 [.skill](#)。其目标不是 增加说明文档数量，而是提高 AI 编码智能体在一次主评测机会中完成陌生算子任务的 概率：先建立仓库与语义基线，再定位最近的 arrangement、application、tensor meta-operation、load/store、wrapper、export 与测试模式，实施最小生产代码补丁，用 PyTorch 或仓库参考实现完成 correctness 闭环，最后才进入 benchmark、generated source、AOT 或 InfiniCore dispatch。

最终包包含 5 个自测案例，其中 4 个覆盖赛题规定的逐元素/广播、归约/分块、布局 敏感、性能/诊断/集成类别，第 5 个是实际修改 production wrapper 并有 GPU 测试和 clean apply-check 的实现型案例。包内保留 add 与 softmax 两份真实短 benchmark CSV，同时明确不声称 broad speedup、full non-contiguous support、AOT、generated source 或 InfiniCore dispatch 已验证。

2. 赛题理解与设计原则

官方规则将隐藏任务实际完成效果作为核心，而不是只检查 [SKILL.md](#) 是否完整。每个 任务只有一次主评测机会，因此本 skill 采用以下原则：

1. 实现优先：用户要求修改时，智能体不能停留在审查或建议。
2. 仓库原生：先读当前仓库规则与相邻实现，复用本地 helper 和测试风格。
3. 最小补丁：不做无关重构，不默认修改 NineToothed 编译器核心。
4. correctness 优先：性能计时必须由相同输入上的 correctness PASS 解锁。
5. 证据分层：区分 [\[VERIFIED\]](#)、[\[INFERRED\]](#)、[\[TODO-GPU\]](#) 与 [\[BLOCKED\]](#)。
6. 失败闭环：保留首个有效错误，确认根因，做最小修复，并重复原命令。
7. 可移植交付：patch 使用 LF、仓库相对路径，并在记录的 clean revision 上验证。
8. 诚实比较：baseline 的优点和 skill-enabled 的不足都写入报告。

3. 适用与不适用范围

适用范围：

- NineToothed/ntops 算子 kernel、wrapper、export 和 correctness test。
- elementwise、broadcast、mask、dtype、reduction、blocking 与数值稳定性。
- stride、storage offset、padding、dilation、source view 和受限非连续布局。
- failing test、patch applicability、generated source、AOT 与性能回退诊断。
- InfiniCore [use_ntops](#)、设备 guard、fallback 和 public-call dispatch 验证。

不适用范围：

- 与 NineToothed 无关的普通 Python、Web、数据分析或文案任务。
- 未经明确要求的编译器核心重构。
- 依赖私有账号、密钥或不可公开在线服务的工作流。
- 未运行却要求写成成功的 GPU、AOT、generated source 或 dispatch 结论。

4. 包结构

```
skills/competition/ninetoothed-operator-dev-skill/
SKILL.md
agents/openai.yaml
README.md
README.zh-CN.md
HONOR_CODE.md
REFERENCE.md
PR_DESCRIPTION.md
SUBMISSION_CHECKLIST.md
SUBMISSION_COMMANDS.md
references/
scripts/
examples/selftests/
tests/
reports/
```

SKILL.md 保留一次执行所需的核心闭环；10 份 reference 按任务类型渐进加载；脚本 负责 deterministic validation、patch 检查、source manifest、短 benchmark 与 dispatch probe；自测目录保留小型原始 CSV、patch、日志摘录和统一 README；测试 只通过 `Path(_file_)` 定位包根目录，不依赖开发工作区。

5. 一次成功导向的核心 workflow

1. 用 Git 和 `rg` 自动发现仓库、revision、规则、相关代码与测试。
2. 写需求卡：语义、shape、dtype、broadcast/mask、axis、边界、layout、stride、offset、容差、不支持项与验收命令。
3. 同时阅读最近的 kernel、wrapper、export、test 和 example。
4. 选择最小实现，修改生产代码和必要测试，避免格式化噪声。
5. 先做 import/collection，再跑最小 correctness，再跑相关测试族。
6. 失败时分类、最小化、验证一个根因、修复并重跑原命令。
7. 性能任务先过 correctness，记录 device、shape、dtype、layout、warmup、repeat、原始 timing 与 caveat。
8. generated source/AOT/dispatch 分别验证产物、build/load、route、correctness 与 fallback，不能以“发现文件”代替成功。
9. 从目标仓库根生成 LF patch，并在目标 revision 上运行 normal/strict apply-check。
10. 最终只输出改动、真实测试、性能、失败、证据路径和剩余风险。

6. Elementwise/Broadcast 自测

案例：SELFTEST-EW-001。

任务围绕 `ntops add` 和 `relu` 的 shape、dtype、alpha、inplace 与边界测试。baseline 补丁更小，偏结构检查；skill-enabled 补丁对 runtime-facing case 和证据组织更完整。两次本地 fresh session 都受 Windows Triton/CUDA 环境限制，未把 skip 或 import 失败写成 correctness PASS。

保存的历史服务器公开检查给出 `add 8 passed`、`relu 16 passed`；本次独立服务器 复核将 add、relu、softmax 合并运行并得到 `32 passed`。修复计时隔离缺陷后，add 短 benchmark 在 RTX 4090、float32 contiguous、`1024x1024`、warmup 10、repeat 30 下得到 PyTorch median `0.017408 ms`、ntops median `0.066000 ms`，baseline/candidate ratio 为 `0.263758`。这是选定输入上的真实回退，不外推到其他 broadcast、dtype 或 shape。

7. Reduction/Blocking 自测

案例：SELFTEST-RED-001。

任务围绕 softmax axis、negative dim、singleton、odd reduction length、dtype 与 numerical stability。baseline 提供了更丰富的 wide-value 和 float64-output case，这是其明确优点；但保存的 baseline patch 未在隔离服务器 workspace 应用，因此没有执行 baseline GPU correctness。

skill-enabled patch 更小并成功应用，focused softmax pytest 为 12 passed in 8.22s。随后才运行短 benchmark：64x1024、float32 contiguous、warmup 10、repeat 30，PyTorch median 0.023568 ms，ntops median 0.071728 ms，本次 baseline/candidate ratio 0.328575。ntops 在该输入上的中位时延约为 PyTorch 的 3.04 倍；这是 selected-shape 回退，不扩展为普遍性能结论。

8. Layout-Sensitive 自测

案例：SELFTEST-LAYOUT-001。

baseline 实际修改 avg_pool2d 与 max_pool2d wrapper，将整数 kernel_size 规范化为二维 tuple，并增加两份测试和共享输入 helper。skill-enabled 保持 test-only，覆盖 deterministic source view、stride、padding、dilation、output shape 与 boundary。

历史隔离服务器结果：baseline patch 应用成功；max pooling 为 62 passed, 54 skipped, 1 xpassed，average pooling 为 26 passed, 18 skipped, 1 xpassed；skill-enabled raw patch 当时在服务器 apply-check 失败。最终独立复核在精确 commit 上发现 skill-enabled helper 的 float16 转换会把非连续 view 物化为连续张量；调整为先转换 dtype/device、再切片后，GPU 结果为 80 passed, 72 skipped。

最终 patch 在 ntops 6bc90d5 上通过 normal/strict apply-check 与 GPU 测试；历史 raw-copy 失败和首次审计的 2 failed, 78 passed, 72 skipped 仍保留在独立日志中。conv2d 仅 diagnostic，本次未运行；不声称 full non-contiguous support。

9. Performance/AOT/Diagnosis 自测

案例：SELFTEST-PERF-AOT-001。

baseline 优点是扫描范围广、文档补丁风险低，并在服务器 apply 成功；缺点是没有改变 runtime。skill-enabled 修改 InfiniCore silu.py，为 ntops fast path 增加 operator 存在性和 AttributeError fallback；该修改更贴近 runtime，但 fresh session 没有补 correctness test，且历史 raw patch 在服务器未应用。

服务器可以导入 Torch 2.6.0a0+ecf3bae40a.nv25.01、Triton 3.1.0、NineToothed 和 ntops，CUDA 12.8 可见，但 infinicore 因缺少 infinicore.lib 导入失败。因此 use_ntops、SiLU dispatch 与 correctness 被阻断，没有运行 benchmark，也没有运行 AOT build。两份 patch 在 InfiniCore d2758a5c 上完成 LF strict apply-check；最终独立 stub 复核通过 missing-operator fallback、ntops fast path 与 AttributeError fallback，但真实 native dispatch 仍保持未验证。

10. 真实实现型自测

案例：SELFTEST-IMPL-001。

该案例直接复用 layout baseline 的真实生产补丁，并明确标记来源，不将 baseline 成果冒充为 skill-enabled 成果。补丁涉及 5 个文件、189 行新增，其中 production 改动只有两个 wrapper 各 3 行；其余为 focused correctness 和 helper。

补丁在目标 clean revision 上通过 git apply --check 与 strict whitespace 检查，最终独立服务器合并结果为 88 passed, 72 skipped, 2 xpassed。这满足“production code + correctness test + applicable patch”的实现型证据要求，同时保留两个限制：未覆盖任意非连续布局，未做性能计时。

11. Benchmark 证据

任务	Operator	输入	Warmup/Repeat	PyTorch median	ntops median	结论
EW	add	1024x1024, float32, contiguous	10/30	0.017408 ms	0.066000 ms	单点回退, ratio 0.263758
RED	softmax	64x1024, float32, contiguous	10/30	0.023568 ms	0.071728 ms	单点回退, ratio 0.328575

两份记录均使用 NVIDIA GeForce RTX 4090。CSV 记录 correctness status **PASS**、mean/median/min、30 个原始样本、device 与版本。计时函数已独立调用 baseline 与 candidate。未运行 long benchmark；未将 compile time 与 steady-state 混合；未声称 官方隐藏 benchmark 或跨 shape/dtype 的普遍性能优势。

12. 失败诊断与修复闭环

Patch 行尾与上下文

layout skill-enabled 与 PERF/AOT baseline 历史 artifact 含 CRLF 风险。最终化时先 备份原文件，再只做 UTF-8 LF 机械规范化，在记录 revision 上运行 normal/strict apply-check。四份 comparison patch 当前均通过，且日志记录 revision、SHA-256、changed files、CRLF count 与 return code。

服务器 patch 失败

历史服务器上 layout skill-enabled 与性能集成 skill-enabled patch 都未应用。报告 保留该事实，并把它归类为 applicability/source-context handoff failure，而不是 runtime correctness failure。后续本地 PASS 只证明记录 revision 上的当前 artifact 可应用。

Layout helper 失败与修复

最终服务器复核首次得到 2 failed, 78 passed, 72 skipped。两项失败都来自测试 helper 在切片后调用 .to(float16)，导致非连续 view 被物化。最小修复是先完成 dtype/device 转换，再执行 [..., ::2] 切片；重跑得到 80 passed, 72 skipped。

InfiniCore native import

ModuleNotFoundError: No module named 'infinicore.lib' 阻断 dispatch。因为 build、load、public call 和 correctness 链路没有建立，AOT、dispatch 与 timing 都停止，没有从 repo scan 推导成功结论。

13. No-Skill 与 Skill-Enabled 对比

自测	Baseline 优点	Skill-enabled 优点	最终判断
EW	补丁更小、结构检查直接	需求卡、失败记录和证据更系统	流程提升明确，fresh session 未证明 runtime 优势
RED	stability/dtype 覆盖更广	patch 应用、12 个测试通过并解锁 benchmark	skill-enabled 执行闭环更完整
LAYOUT	production change 与 GPU correctness 最强	test-only 更窄，修复后 80 passed	baseline 实现更强，skill 测试补丁现可执行
PERF/AOT	扫描广、文档改动低风险、服务器可应用	三个 fallback/fast-path stub 分支通过	双方无 native dispatch/AOT/timing 成功

因此，本 skill 的可信优势是需求提取、证据纪律、correctness gate、失败分类和最终 handoff 规则；不能据此声称每个任务都优于 baseline。最终版已把“必须实施生产改动”和“final patch 必须 clean apply-check”前置为核心规则，针对暴露出的弱点修复指导。

14. 安全、依赖、授权、引用与 AI 披露

- 最终包无服务器凭据、私有路径、隐藏答案、任务名 bypass、评测检测或删除逻辑。

- validator/测试只用 Python 标准库；PyTorch、Triton、NineToothed、ntops 与 InfiniCore 均为按需外部依赖或 target repo，未随 ZIP 打包。
- ntops revision/Apache-2.0 与 InfiniCore revision/MIT License 已核对。
- OpenAI Codex 参与检查、改写、测试、证据和报告；实测计数与 timing 来自 artifact 或实跑命令。参赛者已签署 [HONOR_CODE.md](#)，仍须最终人工审阅，披露见 [REFERENCE.md](#)。

15. 局限性与维护计划

当前局限：

- 没有隐藏任务结果、full non-contiguous support、generated-source runtime artifact、AOT build output 或真实 InfiniCore dispatch 结论。
- benchmark 仅两个 selected-shape、single-GPU、短时记录；PERF/AOT runtime patch 只有 stub 分支测试，没有 native correctness。
- 参赛者身份、团队、GitHub ID、签字和 team-based PDF 文件名已完成；push、PR 和平台上传仍需人工执行。

维护计划：

1. 上游 revision 改变后更新 reading route，并重验 patch 与 production test。
2. 新增 operator/layout case 时保留 before/after test 与明确的布局支持边界。
3. native import 可用后再验证 dispatch；benchmark 扩展时保留矩阵、原始样本与方差；每次发布重复原目录和 clean extraction 验证。

16. 评分项追踪矩阵

评分维度	本包设计与证据	边界
隐藏任务完成度	implementation-first workflow、需求卡、最近模式、最小补丁、失败闭环	无隐藏任务结果，不预估分数
测试与验证	self-contained tests、PyTorch oracle、focused-to-family progression、patch apply-check	GPU 仅引用真实已保存结果
性能材料质量	2 份含 30 个原始样本的 CSV、correctness gate、可复算 timing metadata、保守回退结论	不运行 long benchmark，不外推普遍性能
自测材料质量	4 类规定案例 + 1 个真实实现案例，统一 README 与小型证据	PERF/AOT runtime 仍 blocked
工程可用性	标准 frontmatter、 agents/openai.yaml 、10 份渐进 reference、9 个 CLI、clean ZIP tests、签名版身份信息	push、PR 与平台上传仍需人工执行
泛化能力	elementwise、reduction、layout、failure、benchmark、generated/AOT/dispatch 通用规则	不硬编码隐藏题或评测脚本
补丁最小性与风格	target-root generation、diff checks、LF、clean revision apply-check	不修改编译器核心
过程与合规	secret/claim/link scans、Honor Code、Reference、AI disclosure	最终人工审阅仍必需

结论

最终包已可离线复验，并保留真实 benchmark、失败证据与 baseline 优势。签名版已完成参赛者身份、签名和正式 PDF 命名；后续只剩人工 push、PR 创建和平台上传。